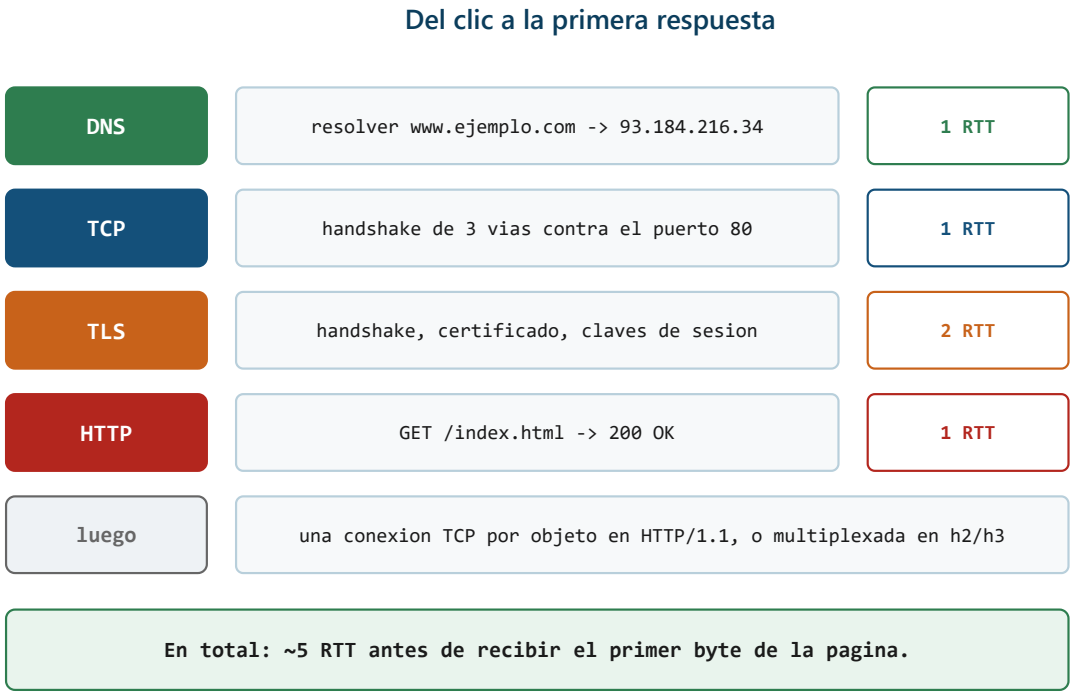
Clase 8 — Modelo cliente/servidor y HTTP/1.1

Bloque: B (Aplicación) · **Semana:** 3 · **Duración estimada:** 110 min **Correlativa:** Clase 7 (la capa de aplicación: peer-to-peer y cliente/servidor) · **Sigue con:** Clase 9 (HTTP a fondo: mensajes, cookies, caché y versiones) **Material de referencia:** Tanenbaum cap. 7 (§7.1 el modelo cliente/servidor, §7.3 HTTP) · RFC 9110, RFC 9112, RFC 9113, RFC 9114, RFC 6265, RFC 7540 (obsoleto), RFC 7838



🎯 Al terminar esta clase vas a poder

1. Explicar qué es el modelo **cliente/servidor**, por qué existe y en qué se diferencia del **peer-to-peer**, y por qué la asimetría (servidor pasivo, cliente activo) simplifica toda la capa de aplicación.
2. Enumerar **qué es HTTP y qué NO es**: por qué no es la Web, por qué no es un protocolo de transporte, por qué no tiene estado y por qué no es seguro por sí mismo.
3. Escribir **de memoria** un mensaje de petición y uno de respuesta HTTP/1.1 completos, con todas sus cabeceras, y explicar qué significa cada código de estado.
4. Calcular **cuántos RTT** cuesta una página con **1 objeto** y con **11 objetos**, en HTTP no persistente y en HTTP persistente, con un solo servidor y con varios, justificando cada RTT.
5. Explicar por qué **304 Not Modified** existe, cómo se combine con `ETag` y `Last-Modified`, y por qué ahorra transferencia.
6. Explicar qué son las **cookies**, qué problema resuelven, dónde vive el estado y por qué `RFC 6265` las estandariza.
7. Resolver completo el **Examen 2010-07 Pregunta 1** (8 puntos), con la cuenta de RTTs.

> **Prerequisito del examen:** `2010-07 P1` (8 puntos — ★ el problema central de esta clase), `2009-07 Problema 2` (30 puntos — cliente anti-spam vía HTTP, parte b), `2010-02 P1` (qué es DNS, misma lógica conceptual).

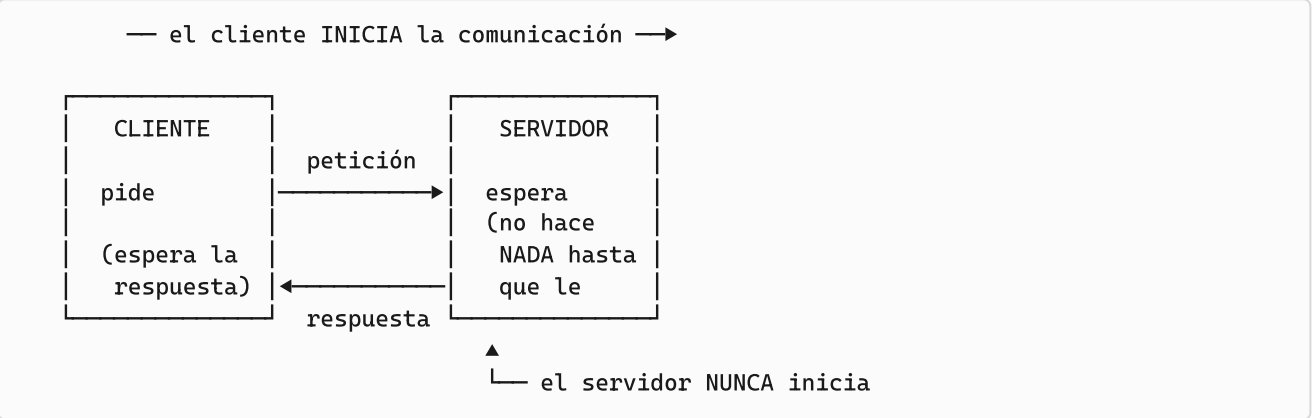
PARTE 1 — TEORÍA

1.1 El modelo cliente/servidor

Qué es

El modelo **cliente/servidor** (*client/server*) es una organización de la computación en la que los procesos se dividen en **dos roles que no son simétricos**:

- El **servidor** (server) es un proceso que **espera**. Ofrece un servicio sobre un puerto bien conocido y no hace nada hasta que alguien le hable.
- El **cliente** (client) es un proceso que **inicia**. Decide hablar, contacta al servidor y consume el servicio.



La asimetría es el dato de diseño central. Un servidor nunca genera tráfico espontáneo hacia un cliente: **solo responde**. Un cliente solo genera tráfico porque **quiso**.

Por qué existe

La razón profunda es la **asimetría de roles entre los extremos**. La Web no es simétrica: hay millones de navegadores (que cambian, se mueven, se apagan) y hay decenas de miles de servidores de contenido (que son máquinas con costos fijos, permanecen y son caros de mantener). Un diseño donde cualquiera pueda servir y servirse sería insostenible: los roles no son intercambiables.

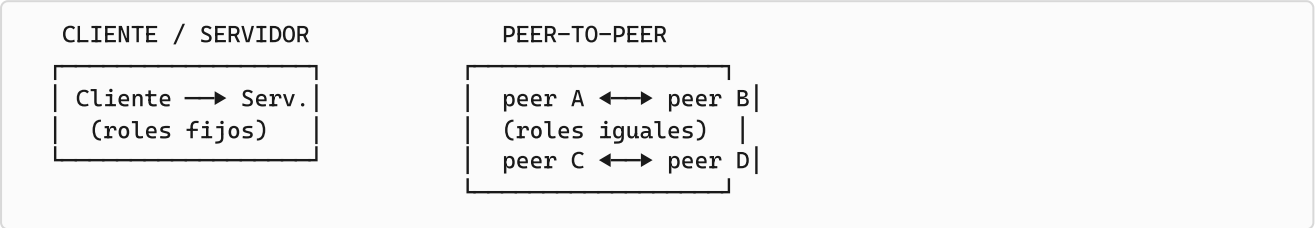
El modelo cliente/servidor **externaliza** el problema: en vez de que cada host mantenga una copia de todo el contenido, el contenido vive en **un** servidor y los clientes van a buscarlo. Las tres consecuencias prácticas que hacen que gane:

Consecuencia	Por qué importa
El servidor no necesita saber quién es el cliente	Basta con direccionar a un `host:puerto`. No hay que "descubrir" al cliente.
El servidor es pasivo ⇒ no hay que resolver el "quién llama"	El problema de "quién está llamando" desaparece: el que llama se identifica solo al abrir la conexión.
Se pueden atender muchos clientes desde un servidor	Un servidor HTTP atiende miles de conexiones simultáneas; los clientes son desechables.

La tercera consecuencia es la que habilita la **Web** como la conocemos: si cada visitante tuviera que publicar su propio contenido desde su casa, no habría nada que ver.

Cliente/servidor vs. peer-to-peer

El **peer-to-peer** (*P2P*, pares) es el modelo opuesto: **todos los nodos son simultáneamente clientes y servidores**, y no hay un servidor distinguible. Es el modelo de BitTorrent, de Skype en su versión original, y de las redes de compartimiento de archivos.



	Cliente/servidor	Peer-to-peer
Roles	fijos y distintos	todos hacen de todo
¿Quién inicia?	el cliente	cualquiera
Centralización	hay un punto central (el servidor)	ninguno
Escala	agregar clientes es barato; el **servidor es el cuello de botella**	escala con el número de pares
Si el servidor cae	el servicio cae	el servicio sigue
Ejemplos	Web, correo, DNS	BitTorrent, Skype, IPFS

Ojo con el ejemplo del examen: el correo electrónico es cliente/servidor aunque el cliente "mande" el correo al servidor. Mandar es **iniciar**, no servir.

> **Por qué importa en el examen:** el `2010-07 P1` es enteramente cliente/servidor y la cuenta de RTTs sale de ahí. Y el `2010-12 Problema 1` (el `connectCheaper()`) es un ejemplo híbrido interesante: los **APs son servidores** (hacen broadcast de precios, un comportamiento de *push* típico de servidor) y el usuario es cliente que **inicia** el CONNECT.

1.2 HTTP: qué es y qué NO es

HTTP = HyperText Transfer Protocol. Es un protocolo de **capa de aplicación** que transfiere recursos (objetos) entre un cliente y un servidor, con el esquema petición/respuesta. Para entenderlo bien, lo más útil es la lista de lo que **no** es, porque cada "no es" explica una pregunta de examen.

(a) HTTP no es un protocolo de transporte

HTTP **no** garantiza entrega, no numera, no retransmite, no ordena y no controla la congestión: todo eso lo hace **TCP**, debajo. Corolario práctico: **HTTP no sabe si un mensaje llegó.** Si se pierde, TCP lo reintenta por abajo y HTTP nunca se entera.

(b) HTTP no es lo mismo que la Web

```
La WEB  =  un conjunto de TECNOLOGÍAS
   ├── un espacio de nombres:   los URL (y el DNS que los resuelve)
   ├── un protocolo de transferencia:  HTTP
   └── un formato de documento:         HTML

HTTP    =  SOLO el protocolo de transferencia de esos documentos
```

Decir "voy por la Web" mezcla tres capas distintas. Si te preguntan "qué es HTTP", la respuesta empieza por "es **un** protocolo de capa de aplicación para transferir objetos identificados por URL".

(c) HTTP no tiene estado

Sin estado (*stateless*) significa: **cada petición se atiende sola, sin memoria de las anteriores**.

```
Cliente                              Servidor
   |  GET /carrito                     |
   |──────────────────────────────────►|  ¿Y qué compró el usuario 17?
   |                                   |   NO SÉ: HTTP es sin estado
   |  404 (o un carrito vacío)         |
   |◄──────────────────────────────────|
   |  GET /carrito?user=17             |   ← el estado va en la URL
   |──────────────────────────────────►|  (o en una cookie)
   |  200 {"camisa": 2}                |
   |◄──────────────────────────────────|
```

Esto es una decisión de diseño, no una limitación. Un protocolo con estado obligaría al servidor a almacenar quién pidió qué, y eso rompe el balanceo de carga: si el estado está en el servidor, todas las peticiones de un usuario tienen que ir al **mismo** servidor y no podés escalar horizontalmente. HTTP prefirió **stateless** y corrió el riesgo de que el estado viaje en la petición. Es el mismo compromiso que "DNS versus NAT": simplicidad a cambio de eficiencia.

(d) HTTP no es seguro, y viaja sobre TCP

En texto plano HTTP viaja **tal cual** por la red: cualquiera en el camino ve las cabeceras `Authorization`, `Cookie` y los formularios con tu contraseña. Para eso está **HTTPS**, que es HTTP sobre **TLS**: **HTTP aporta semántica, TLS aporta confidencialidad, integridad y autenticidad** (la Clase 47 muestra TLS entero).

Y HTTP **no viaja sobre UDP**: viaja sobre **TCP, puerto 80** (o **443** con TLS). Eso es lo que lo hace fiable y lo que lo hace lento: la fiabilidad tiene un costo de setup.

1.3 El mensaje HTTP: la petición

Una petición HTTP/1.1 tiene **tres** partes, en este orden y sin falta:

```
línea de petición (request line)  +  cabeceras  +  línea en blanco  +  cuerpo
──────────────────────────────────    ──────────    ─────────────────    ──────
GET /index.html HTTP/1.1              Host: ...                          (puede
<CR><LF>                              Accept: ...                        ir vacío)
<CR><LF>
```

La línea en blanco final es obligatoria, aunque el cuerpo esté vacío. Sin ella el servidor no sabe dónde termina la cabecera. Esto es el mismo principio del **framing** de la capa 2: hay que delimitar.

Un ejemplo **real y completo**, tal como lo manda un navegador:

```
GET /index.html HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Encoding: gzip
Connection: keep-alive
```

Fijate que **la última línea es vacía**: ese es el `CRLF` extra que cierra las cabeceras. Sin esa línea en blanco, HTTP no parsea.

Desglose de la línea de petición, campo por campo:

Campo	Significado	Ejemplo
Método	qué quiere el cliente	`GET`
URI (*Uniform Resource Identifier*)	qué recurso	`/index.html`
Versión	qué dialecto habla el cliente	`HTTP/1.1`

Y de las cabeceras que aparecen:

Cabecera	Para qué sirve	Por qué importa
`Host`	**obligatoria** en HTTP/1.1: dice a qué **servidor** dentro del destino querés llegar	Sin ella, un servidor que aloja miles de sitios (como un CDD) no sabe cuál teSirve. Es el motivo de que HTTP/1.1 no pueda esconderse tras NAT.
`User-Agent`	identifica el cliente	Los servidores usan esto para decidir qué versión mandarte.
`Accept`	qué **tipos de contenido** aceptás	`text/html`, `image/png`
`Accept-Encoding`	qué **codificaciones** de compresión aceptás	`gzip`, `br`. El servidor responde `Content-Encoding: gzip` si lo comprime.
`Connection: keep-alive`	el cliente pide **reusar** la conexión	En HTTP/1.1 es el default, pero lo digo explícito para que no se note.

Los métodos: seguros e idempotentes

Seguro = no cambia el estado del servidor. **Idempotente** = si lo repetís *n* veces el efecto es el mismo que si lo hicieras una. `GET`, `HEAD`, `OPTIONS` y `TRACE` son **seguros**; `GET` y `HEAD` son además **idempotentes**, igual que `PUT` y `DELETE`. `POST` **no es ni seguro ni idempotente**.

Por qué importa. Un cliente que reintenta por timeout necesita saber si puede reintentar. Si la respuesta se perdió pero el servidor sí la procesó, reintentar un `POST` duplica el cobro; reintentar un `GET` no pasa nada. La Clase 9 desarrolla métodos, seguridad e idempotencia en detalle.

1.4 El mensaje HTTP: la respuesta

```
línea de estado (status line)  +  cabeceras  +  línea en blanco  +  cuerpo
```

Un **200 OK** completo:

```
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 14:22:31 GMT
Server: Apache/2.4.62 (Ubuntu)
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
ETag: "3f2a1c8-9b41-6512d3f0"
Cache-Control: max-age=600
Content-Type: text/html; charset=utf-8
Content-Encoding: gzip
Content-Length: 4096
Accept-Ranges: bytes
Connection: keep-alive

<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>FING</title></head>
<body>...</body>
</html>
```

La línea de estado son tres cosas separadas por espacio:

Parte	Qué es	Ejemplo
Versión	el dialecto que devuelve el servidor	`HTTP/1.1`
Código	número de 3 dígitos	`200`
Frase de razón	texto legible, decorativo	`OK`

Las dos cabeceras que hacen que esto funcione y que hay que mirar siempre:

- `Content-Length`: cuántos bytes tiene el cuerpo. Es lo que le permite al cliente saber dónde termina la respuesta **sin** tener que cerrar la conexión.
- `Content-Type`: qué **tipo** es el cuerpo (`text/html`, `image/png`). Sin esto el cliente no sabría si tiene que mostrarlo, guardarlo o ejecutarlo.

Un 304 Not Modified (y por qué existe)

```
HTTP/1.1 304 Not Modified
Date: Tue, 06 Oct 2026 14:22:33 GMT
ETag: "3f2a1c8-9b41-6512d3f0"
Cache-Control: max-age=600
Connection: keep-alive
```

Fijate que NO hay cuerpo y NO hay `Content-Length`. Eso es correcto: un 304 significa literalmente "el recurso **no cambió**, lo que tenés guardado sirve". El cuerpo no viaja porque no hay nada nuevo. Por eso el `304` no consume ancho de banda de contenido: gasta **solo** las cabeceras.

¿Por qué existe el 304? Porque descargar un archivo de 4 MB para descubrir que es idéntico al que ya tenías es absurdo. El 304 es el mecanismo de **validación condicional**: el cliente dice "lo tengo, etiqueta `3f2a1c8`, ¿sigue siendo cierto?" y el servidor responde "sí, te sirve" con 30 bytes de cabeceras en lugar de 4 MB de datos. La clase 9 lo desarrolla con el intercambio completo.

Un 404 Not Found

```
HTTP/1.1 404 Not Found
Date: Tue, 06 Oct 2026 14:22:31 GMT
Server: Apache/2.4.62 (Ubuntu)
Content-Type: text/html; charset=utf-8
Content-Length: 196
Connection: keep-alive

<html><body><h1>404 - Not Found</h1></body></html>
```

El **404** significa "el servidor entendió tu petición pero **no tiene** ese recurso". Importante: el 404 **no dice que la URL esté mal escrita** (eso sería un **400 Bad Request**). El 404 dice "no existe, pero el servidor está vivo y te contesta". Por eso es distinto de un **500** (el servidor se rompió) o de un **502/504** (un proxy no pudo hablar con el backend).

1.5 Los códigos de estado: las cinco clases

El primer dígito del código es la **clase**, y la clase **te dice el tipo de problema** sin leer el número:

Clase	Rango	Significado	Ejemplos que hay que saber
1xx	100–199	**informativo**: "seguí, todavía no terminé"	`100 Continue`, `101 Switching Protocols`
2xx	200–299	**salió bien**	`200 OK`, `201 Created`, `204 No Content`, `206 Partial Content`
3xx	300–399	**redirección**: "andá a otro lado"	`301`, `302`, `304 Not Modified`, `307`, `308`
4xx	400–499	**error del cliente**: "vos te equivocaste"	`400`, `401`, `403`, `404`, `405`, `408`, `429`
5xx	500–599	**error del servidor**: "yo me rompí"	`500`, `502`, `503`, `504`

Los que más aparecen y qué significan de verdad: `200` (acá está, te lo mando acá), `201 Created` (lo creé, te digo dónde, en `Location`), `204 No Content` (salió bien pero **no hay cuerpo**), `206 Partial Content` (te mando **parte**, respuesta a un `Range`), `301` (te mudaste **para siempre**), `302` (mudate **por ahora**), `304` (no te lo mando, conservá el tuyo), `401` (**no estás autenticado**), `403` (**no podés**, ni autenticándote), `404` (no existe), `429 Too Many Requests` (te mandaste demasiadas), `500` (el servidor se rompió), `502`

`Bad Gateway` (un proxy no pudo hablar con el backend), `503` (caído o sobrecargado) y `504 Gateway Timeout` (el proxy esperó demasiado).

El par 401/403 es la pregunta clásica. Ambos son 4xx y ambos parecen "no autorizado", pero significan cosas distintas:

```
401  → "NO SÉ QUIÉN SOS. Mandame credenciales (y reintentá)."
403  → "SÉ QUIÉN SOS, y SÍ, no tenés permiso. Reintentar no sirve."
```

`401` es recuperable; `403` no lo es. Confundirlos es un error de examen muy común.

1.6 HTTP no persistente vs. persistente — el conteo de RTT

Esta es **la** pregunta del examen. Vamos a hacer la cuenta exacta, con los números a la vista.

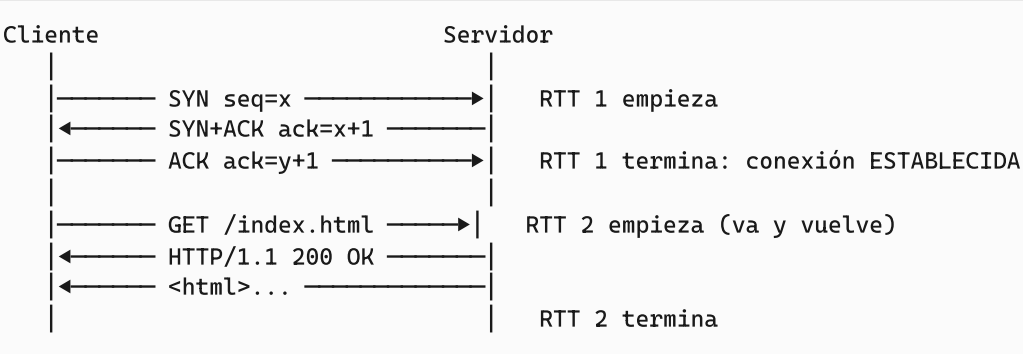
Las dos definiciones, sin ambigüedad

- **HTTP no persistente** (*non-persistent*): por cada objeto, **se abre una conexión TCP nueva, se pide, se recibe, y se cierra**. Una conexión = un objeto.
- **HTTP persistente** (*persistent*): **se abre una conexión TCP y se mantiene** para pedir **varios** objetos seguidos, uno tras otro, en la **misma** conexión.

El término clave es **"cuántas conexiones TCP"**. No es "cuántas peticiones": en persistente hacés 11 peticiones igual, pero sobre **una sola** conexión.

El caso base: 1 objeto, 1 RTT de handshake

Toda conexión TCP nueva cuesta **1 RTT** antes de poder mandar datos. El handshake de 3 vías es:



Entonces, **1 objeto**:

```
No persistente:   1 RTT (TCP)  +  1 RTT (petición/respuesta)  =  2 RTT
Persistente:      1 RTT (TCP)  +  1 RTT (petición/respuesta)  =  2 RTT
```

Con 1 objeto no hay diferencia. Es el caso trivial y hay que decirlo, porque la respuesta "no persistente es peor" sin matices es incorrecta para 1 objeto.

El caso de 11 objetos (1 HTML + 10 imágenes), todo en el MISMO servidor

No persistente: cada objeto paga su propio handshake.

```
objeto 1 (HTML):    1 RTT TCP  +  1 RTT HTTP  =  2 RTT
objeto 2 (img 1):   1 RTT TCP  +  1 RTT HTTP  =  2 RTT
objeto 3 (img 2):   1 RTT TCP  +  1 RTT HTTP  =  2 RTT
...
objeto 11 (img 10): 1 RTT TCP  +  1 RTT HTTP  =  2 RTT

Total = 11 objetos × 2 RTT = 22 RTT
```

Persistente: un solo handshake, y después 11 RTT de HTTP (uno por objeto).

```
conexión TCP (una sola):            1 RTT
objeto 1 (HTML):                    1 RTT
objetos 2..11 (10 imágenes):        10 RTT
                                    ─────

Total = 1 + 11 = 12 RTT
```

Comparación: `22 RTT` vs. `12 RTT`. **El persistente es 10 RTT más rápido**, que es exactamente el `N − 1 = 10` handshakes que se ahorró. El HTML y las 10 imágenes van por la misma tubería.

```
No persistente                      Persistente
──────────────                      ───────────
[TCP][HTML]                          [TCP]──[HTML]──[img1]──[img2]── ... ──[img10]
 2 RTT                               1 RTT      1 RTT   1 RTT             1 RTT
×11                                 ──────────────────────────────────────────────
= 22 RTT                            = 12 RTT
```

Y si los objetos vienen de varios servidores

Este es el matiz que casi nadie menciona y el que el apartado b) del examen pide **discutir**. Supongamos que el HTML viene del servidor A y que las 10 imágenes están repartidas en 3 servidores (A, B, C).

Secuencial (el navegador no paraleliza):

```
Persistente, 3 servidores:   3 handshakes (uno por servidor) + 11 RTT de HTTP
                              = 3 + 11 = 14 RTT

No persistente:              11 objetos × 2 RTT = 22 RTT
```

En paralelo (que es lo que hacen los navegadores reales: abren 6 conexiones por host):

```
HTML por A:                1 RTT TCP + 1 RTT HTTP = 2 RTT
las 10 imágenes, en paralelo:
   los 3 handshakes, simultáneos:   1 RTT   (no 3: van a la vez)
   las 10 peticiones, simultáneas:  1 RTT
                                    ──────
Persistente, 3 servidores, PARALELO   = 2 + 2 = 4 RTT
No persistente, 3 servidores, PARALELO = 2 + 2 = 4 RTT   ← ¡empate!
```

Conclusión honesta del apartado b). El persistente **no siempre** es mejor:

```
Secuencial (1 servidor):   no-pers 22 RTT | pers 12 RTT   → gana el pers
Secuencial (3 servidores): no-pers 22 RTT | pers 14 RTT   → gana el pers
Paralelo   (1 servidor):   no-pers  4 RTT | pers  3 RTT   → gana el pers
Paralelo   (3 servidores): no-pers  4 RTT | pers  4 RTT   → empate
```

Y hay una ventaja extra del no persistente que el enunciado no pide pero que **suma puntos**: el **cliente puede elegir** en qué servidor va cada imagen (por ejemplo el más cercano geográficamente), porque cada conexión es independiente.

1.7 Las estadísticas: por qué HTTP se siente lento

Un resultado clásico y muy reproducible: **la mayor parte del tiempo de carga de una página no es "bajar bytes", sino "esperar al servidor"**. Vamos a hacerlo con números reales, porque es lo que suma puntos.

Datos. RTT desde Montevideo hasta un servidor en Estados Unidos: **100 ms**. Ancho de banda del cliente: **10 Mbps**. La página tiene **11 objetos** de **50 KB** cada uno, servidos en HTTP no persistente desde un solo servidor.

```
TIEMPO DE ESPERA
   = 22 RTT × 100 ms = 2200 ms

TIEMPO REAL DE TRANSFERENCIA
   Total de datos = 11 × 50 KB = 550 KB
   t_trans = 550 KB × 8 bits/KB / 10 Mbps
           = 550 × 1024 × 8 bits / 10·10^6 bits/s
           = 4.505.600 bits / 10.000.000 bits/s
           = 0,4505 s ≈ 450 ms

PROPORCIÓN DE ESPERA
   = 2200 / (2200 + 450) = 2200 / 2650 = 0,83  →  83 %
```

Interpretación: más del 80 % del tiempo se va esperando al servidor, no transfiriendo datos. Y si lo hacés **persistente**, el mismo cálculo da `1200 / (1200 + 450) = 73 %`: mejor, pero el tiempo real de transferencia (450 ms) **no cambió en absoluto**. Cambiar de protocolo no acelera la red: **elimina esperas**.

El tiempo de carga de una página es `espera × número de viajes`. Todo lo que reduce el número de viajes acelera la página: las conexiones persistentes, el **CDN** (que pone un servidor *cerca* para que el RTT sea 5 ms en vez de 100 ms), la resolución DNS cacheada. Y por supuesto HTTP/2 y HTTP/3, que existen exactamente para **matar esperas**. La Clase 9 lo desarrolla.

1.8 Las cookies

Qué son. Una **cookie** (*galletita*) es una pequeña cadena de texto que el **servidor** le manda al cliente y que el cliente **guarda y reenvía** en peticiones posteriores al **mismo** dominio.

```
GET /carrito HTTP/1.1  ──────►  Servidor
◄──────  Set-Cookie: sesion=abc123; Path=/; Max-Age=3600

GET /carrito HTTP/1.1  ──────►  Servidor
Cookie: sesion=abc123
◄──────  200 {"camisa": 2, "pantalon": 1}
```

`Set-Cookie` es la cabecera que **el servidor manda** para **crear o actualizar** una cookie. `Cookie` es la cabecera que **el cliente manda** para **devolverla**. Son dos cabeceras distintas con dos roles distintos, y esa distinción es lo que se pregunta.

El problema que resuelven: el estado en el cliente

HTTP es sin estado, pero casi todas las aplicaciones necesitan recordar algo: el **carrito de compras** (`carrito=camisa:2,pantalon:1`), la **sesión iniciada** (`sesion=abc123`, para no pedir la contraseña cada vez), las **preferencias de idioma** (`lang=es`) o el **"recordarme"** (`token=xyz`).

El estado vive en el cliente. Es el precio de la estatitud: si HTTP no puede guardar estado en el servidor, lo guardás **en el cliente** y lo mandás de vuelta en cada petición. El ejemplo canónico es el carrito:

```
1) El usuario mete una camisa.
   GET /agregar?item=camisa HTTP/1.1
   ◄──────  Set-Cookie: carrito=camisa; Path=/; Max-Age=86400

2) El usuario mete un pantalón.
   GET /agregar?item=pantalon HTTP/1.1
   Cookie: carrito=camisa                          ← el cliente reenvía
   ◄──────  Set-Cookie: carrito=camisa,pantalon    ← el servidor actualiza

3) El usuario mira el carrito.
   GET /carrito HTTP/1.1
   Cookie: carrito=camisa,pantalon                  ← el estado viaja acá
   ◄──────  200 {"camisa": 1, "pantalon": 1}
```

En el paso 3 el servidor **no tiene memoria** de los pasos 1 y 2: todo lo que sabe del usuario está **en la cookie que el cliente le devolvió**.

Los atributos de una cookie

```
Set-Cookie: sesion=abc123; Path=/; Max-Age=3600; Secure; HttpOnly; SameSite=Lax
            │                  │       │          │        │          │
            │                  │       │          │        │          └ no mandar en
            │                  │       │          │        └ peticiones de terceros
            │                  │       │          └ no accesible por JavaScript
            │                  │       └ segundos de vida (si no, "session cookie")
            │                  └ las rutas donde viaja
            └ NOMBRE = VALOR
```

`Secure` restricts el envío a HTTPS. `HttpOnly` la vuelve invisible al JavaScript de la página (defensa contra el robo por XSS).

⚠️ Actualización a 2026 — cookies

Sigue igual: el par `Set-Cookie` (servidor) / `Cookie` (cliente) y que el estado viaje **en el cliente**.

Cambió o se agregó: - `RFC 6265` **— HTTP State Management Mechanism** es el estándar que define el formato. Si el enunciado te pide "qué RFC define las cookies", esa es. - `SameSite`: con `SameSite=Lax` (el default moderno) el navegador **no manda la cookie en peticiones originadas por otros sitios**, lo que corta de raíz el **CSRF** (*Cross-Site Request Forgery*). - **Las cookies de terceros están en retirada.** Chrome (*Privacy Sandbox*), Safari (ITP) y Firefox (ETP) las bloquean o particionan por defecto: eso obliga a los sitios a volver a **sesiones del lado del servidor**. - Con **DoH**, las cookies siguen igual respecto del DNS: son de **capa de aplicación**. Lo que cambia es la privacidad de la resolución.

> **Consecuencia para el examen:** si te preguntan "cómo se rompe el modelo sin estado de HTTP", la respuesta es **cookies** (o equivalentemente `Authorization` con un token). Si te preguntan "dónde vive el estado", la respuesta es **en el cliente**, y por eso el servidor puede ser *stateless* y escalar.

1.9 ⚠️ Actualización a 2026 — la especificación de HTTP que hay que corregir

Sigue igual — de memoria, sin dudar: - Modelo **petición/respuesta**, protocolo **sin estado**. - Los **métodos**: GET, HEAD, POST, PUT, DELETE, OPTIONS, TRACE, CONNECT (+ PATCH). GET/HEAD/OPTIONS/TRACE **seguros**; GET/HEAD **idempotentes**. - Las **5 clases** de códigos de estado (1xx a 5xx) y los del día a día: 200, 201, 204, 206, 301, 302, 304, 307, 308, 400, 401, 403, 404, 405, 408, 429, 500, 502, 503, 504. - `Host` **obligatoria** en HTTP/1.1, y **persistente por defecto**. - `Range` requests y `304 Not Modified`.

Cambió — y esto es lo que hay que corregir si citás material viejo: - El curso de 2012 enseñaba `RFC 2616` (1999). **Hoy la especificación vigente es de 2022**: `RFC 9110` (semántica, *Internet Standard*) + `RFC 9112` (sintaxis de HTTP/1.1). - `RFC 7540` (HTTP/2, 2015) está **OBSOLETA**: la reemplazó `RFC 9113`. - **HTTP/3 (`RFC 9114`)** va sobre **QUIC (`RFC 9000`)**, que **reemplaza a TCP por completo**: no hay handshake de 3 vías ni conexión TCP. Se negocia por `Alt-Svc` (`RFC 7838`), no por `Upgrade:`. - **Cifra real (Cloudflare Radar, Year in Review 2025):** HTTP/2 **50 %**, HTTP/1.x **29 %**, HTTP/3 **21 %**. **HTTP/3 no se adueñó de la web.** La Clase 9 lo explica. - **Wireshark decodifica HTTP/3, QUIC, TLS 1.3 y DoH** built-in.

> **Consecuencia para el examen:** HTTP/1.1 **sigue siendo totalmente válido y necesario**. Si te piden "qué versión de HTTP es la actual": "HTTP/2 es el más usado; HTTP/3 va en crecimiento y no se adueñó; HTTP/1.1 sigue vivo y es el baseline de comparación".

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Contar RTTs para 1 y 11 objetos (★ el del examen)

Enunciado. Un cliente pide una página con **1 objeto** (solo el HTML) y después otra con **11 objetos** (1 HTML + 10 imágenes). El RTT cliente-servidor es de **50 ms**. Todo se sirve desde **el mismo servidor** y en el modo más simple posible (el navegador **no** abre conexiones en paralelo).

a) ¿Cuántos RTT cuesta cada caso en HTTP no persistente?

Razonamiento. En no persistente, **cada objeto abre su propia conexión TCP**. El costo por objeto es `(1 RTT de handshake) + (1 RTT de petición/respuesta)` = 2 RTT.

Ecuación:

```
No persistente = N objetos × 2 RTT

1 objeto:   1 × 2 = 2 RTT   → 2 × 50 ms = 100 ms
11 objetos: 11 × 2 = 22 RTT → 22 × 50 ms = 1100 ms
```

Interpretación: con 1 objeto, 100 ms. Con 11, **11 veces más**: 1100 ms. El HTML tardó lo mismo, pero las 10 imágenes cada una arrastró su propio setup de TCP.

b) ¿Y en HTTP persistente? Un solo handshake para **todos** los objetos, y después 1 RTT de HTTP por objeto.

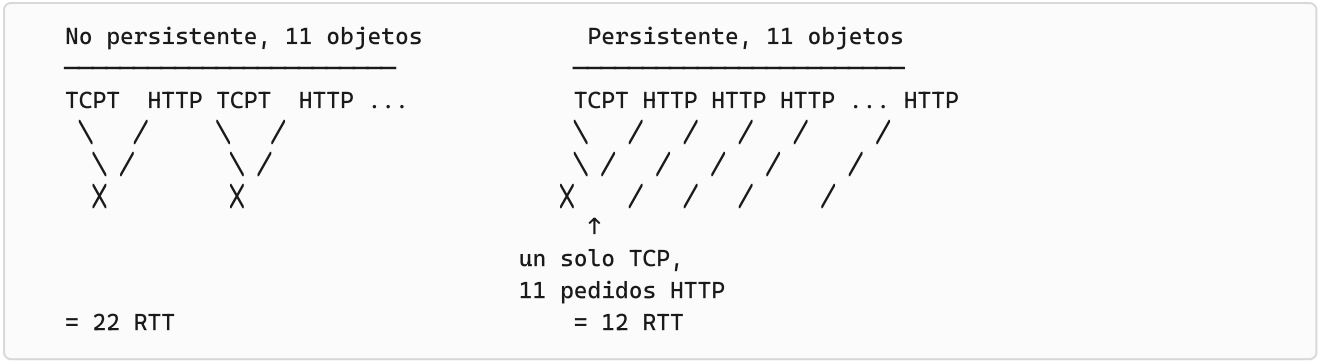
```
Persistente = 1 RTT (TCP) + N × 1 RTT (HTTP)

1 objeto:   1 + 1×1 = 2 RTT  → 100 ms
11 objetos: 1 + 11×1 = 12 RTT → 600 ms
```

Interpretación: el persistente con 11 objetos tarda **600 ms contra 1100 ms**. Ahorró `22 − 12 = 10 RTT = 500 ms`, que es exactamente `N − 1` handshakes eliminados.

c) Comparación final, los cuatro casos (el que se pregunta en b) del examen).

Caso	Modo	Conexiones TCP	RTT	Tiempo (RTT = 50 ms)
1 objeto	no persistente	1	2	100 ms
1 objeto	persistente	1	2	100 ms
11 objetos	**no persistente**	**11**	**22**	1100 ms
11 objetos	**persistente**	**1**	**12**	**600 ms**



d) ¿Y si los objetos vienen de varios servidores? (esto es lo que el apartado b) pide **discutir**)

Supongamos que el HTML viene del servidor A y las 10 imágenes se reparten en **3** servidores (A, B y C).

Razonamiento, caso secuencial. En persistente hay que abrir **una conexión por servidor** (3 handshakes), y encima los 11 objetos viajan por ellas:

```
Persistente, 3 servidores, secuencial
   = (3 servidores × 1 RTT de handshake) + (11 objetos × 1 RTT de HTTP)
   = 3 + 11 = 14 RTT
```

Interpretación: **14 RTT contra 22** del no persistente. Sigue ganando el persistente, pero **menos** (ganaba 10 RTT, ahora gana 8), porque con más servidores hay más handshakes que pagar.

Razonamiento, caso paralelo. Si el navegador abre las conexiones **en paralelo** (que es lo que hacen los reales), los 3 handshakes pasan **al mismo tiempo**, así que cuestan 1 RTT en total, y las 11 peticiones también en paralelo, 1 RTT:

```
Persistente, 3 servidores, PARALELO
   = 1 RTT (los 3 handshakes, en paralelo)
     + 1 RTT (las 11 peticiones/respuestas, en paralelo)
   = 2 RTT

No persistente, 3 servidores, PARALELO
   = 2 RTT (HTML) + 1 RTT (handshakes) + 1 RTT (peticiones) = 4 RTT
```

Interpretación: acá **el persistente y el no persistente empatan en 4 RTT**. Con paralelismo, **la ventaja del persistente se desvanece** cuando hay varios servidores. Por eso el apartado b) dice "puede resultar más performante" y no "siempre es más performante". La respuesta completa al examen es: **el persistente gana en secuencial; en paralelo, gana sólo si hay un solo servidor.**

⚠️ Errores típicos

- **Contar "11 RTT" para el no persistente.** No: son 2 RTT por objeto (uno de TCP + uno de HTTP), así que 22. El error viene de olvidar que **cada objeto abre su conexión**.
- **Decir que el persistente siempre gana.** El enunciado pregunta "¿mejora la performance? **Discuta**". Con conexiones en paralelo y varios servidores, **empata**. Hay que decir las dos caras.
- **Sumar 1 RTT de TCP por servidor en paralelo.** En paralelo, N handshakes simultáneos cuestan **1 RTT**, no N.

- **Olvidar que el `Content-Length` y la línea en blanco final son parte del mensaje.** En un examen de "escribí el mensaje", eso se cuenta.

2.2 Ejercicio 2 — Interpretar un 304

Enunciado. El cliente tiene en su caché `index.html` con `ETag: "3f2a1c8"`, y hace esta petición condicional:

```
GET /index.html HTTP/1.1
Host: www.fing.edu.uy
If-None-Match: "3f2a1c8"
```

Y el servidor responde:

```
HTTP/1.1 304 Not Modified
Date: Tue, 06 Oct 2026 14:22:33 GMT
ETag: "3f2a1c8"
Cache-Control: max-age=600
```

a) ¿Qué le está diciendo el servidor al cliente?

Razonamiento. El cliente mandó `If-None-Match: "3f2a1c8"`, o sea "tengo la versión con esta etiqueta, decime si sigue siendo la actual". El servidor compara su ETag actual con el que le mandaron. Si coinciden, **no manda el cuerpo**.

Respuesta: "lo que tenés guardado sigue siendo correcto, no te mando nada nuevo". El cliente **sigue usando su copia**. Sin esa cabecera condicional el cliente siempre **re-descargaría** todo, porque no tiene forma de decir "esto no te lo pido": el 304 es lo que convierte la caché en algo útil.

c) ¿Cuánto se ahorró?

Razonamiento. El `index.html` tiene 4096 bytes de cuerpo (del 200 del §1.4). El 304 no transmite cuerpo, solo cabeceras.

Ecuación:

```
Con 200 (sin validación):  4096 de cuerpo + ~250 de cabeceras ≈ 4346 B
Con 304:                      0 de cuerpo + ~150 de cabeceras ≈  150 B

Ahorro = 4346 − 150 = 4196 B
Porcentaje ahorrado = 4196 / 4346 ≈ 96,5 %
```

Interpretación: se transfirió el **3,5 %** de lo que se habría transfirido. En una página con 11 objetos, si todos dieran 304, la carga pasa de ~2,2 s a unos pocos cientos de milisegundos. **El 304 no es una optimización de bytes, es un cambio de orden de magnitud.**

2.3 Ejercicio 3 — ¿Por qué HTTP es sin estado y cómo lo resuelven las cookies?

Enunciado. Explicá por qué HTTP fue diseñado sin estado, qué problema concreto genera, y cómo lo resuelven las cookies. Respuesta al estilo del examen.

a) ¿Por qué sin estado? La razón profunda es el **balanceo de carga** (*load balancing*). Si el servidor guardara el estado de cada usuario, **todas las peticiones de ese usuario tendrían que ir al mismo servidor**.

```
CON ESTADO en el servidor:
   user 17 ──► servidor 1 (siempre)    ← si el servidor 1 cae, cae el user

SIN ESTADO:
   user 17 ──► servidor 1, 2 o 3, indistinto  ← sobrevive a la caída de cualquiera
```

Interpretación: el diseño sin estado **sacrificó eficiencia por escalabilidad** y eligió bien: la web creció a miles de servidores detrás de balanceadores, algo que un diseño con estado en servidor hace muy difícil.

b) ¿Qué problema concreto genera? Que el servidor no puede saber quién sos entre peticiones. Si hacés `GET /carrito` sin cookie, no hay forma de saber qué llevás.

c) ¿Cómo lo resuelven las cookies? Poniendo **el estado del lado del cliente** y haciéndolo viajar en cada petición:

```
El estado del carrito NO está en el servidor.
Está en la cookie del cliente:  carrito=camisa,pantalon

El cliente la manda en cada GET /carrito
→ el servidor la lee, responde bien, y la "olvida"
→ el servidor puede ser cualquiera de los 3
```

Alternativas a la cookie (también estado en el cliente): la **URL** (`GET /carrito?user=17` , simple pero queda en el historial y en el log), la cabecera **`Authorization`** con un token Bearer (lo que usa OAuth) y el **`localStorage`** del navegador (estado en el cliente pero solo para el JS).

d) El costo de esta decisión. (esto es lo que suma puntos)

```
Ventaja:  el servidor es stateless, escala, y es tolerante a la caída de
          cualquier nodo.
Costo 1:  el estado viaja en CADA petición → más bytes por request. Si fuera
          grande, la cookie se vuelve pesada y hay que pasar el estado al
          servidor (base de datos), que es lo que hacen las apps grandes.
Costo 2:  la cookie la puede leer o alterar el cliente → hay que firmarla o
          cifrarla, o usar tokens de sesión del lado del servidor.
```

Interpretación final para el examen: **"HTTP es sin estado, y el estado viaja en el cliente (cookies, URL o tokens), por eso el servidor escala, y el precio es que el estado viaja en cada petición y es manipulable por el cliente".**

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-07, Pregunta 1 (8 puntos) — página HTML con 10 imágenes

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf **Este es el problema central de toda la unidad de aplicación.**

Enunciado (texto literal del examen)

> **Pregunta 1 (8 puntos)**
>
> Suponga que un usuario intenta cargar en su browser una página HTML. Esta página contiene texto y diez imágenes.
>
> **a)** Describa la secuencia de pasos que ocurren a nivel de capa de aplicación en el cliente y el servidor a los efectos de desplegar la página completa, suponiendo que se utiliza HTTP no persistente.
>
> **b)** ¿Cuál es la diferencia con HTTP persistente? ¿Mejora la performance? Discuta.

Solución oficial (del PDF, textual)

> a)
>
Paso	Responsable	Acción
1	Cliente	Establece la conexión con el servidor
2	Cliente	Envía Request pidiendo la página
3	Servidor	Envía Response con el contenido de la página
4	Servidor	Cierra la conexión
5*	Cliente	Establece la conexión con el servidor
6*	Cliente	Envía Request pidiendo la la imagen 1
7*	Servidor	Envía Response con el contenido de la imagen 1
8*	Servidor	Cierra la conexión
>
> * Se repiten los pasos 5 al 8 para cada una de las imágenes.
>
> b) En el caso de HTTP persistente, se establece la conexión una sola vez, se obtiene el contenido de la página, y a través de la misma conexión se piden todas las imágenes.
>
> El uso de HTTP persistente puede resultar más performante en los casos en que se utilice una sola conexión por vez (por ejemplo si el servidor nos limita por IP), ya que no existe el overhead de crear

> nuevas conexiones para cada imagen.
>
> Si el navegador abre conexiones en paralelo para obtener varias imágenes a la vez, probablemente termine de obtener todos los datos más rápidamente, aprovechando mejor el ancho de banda.

a) La secuencia de pasos — resolución

La tabla oficial, reproducida tal cual, es la respuesta al apartado a) y está perfectamente bien para los 8 puntos. Pero le agrego dos precisiones que la oficial omite y que suman:

Precisión 1 — el paso 1 "establece la conexión" es el handshake de TCP. El enunciado dice "a nivel de capa de aplicación" y la oficial abstrae el TCP en un solo paso, que es lo que el enunciado pide. **PERO** para contar RTTs hay que saber que ese paso 1 vale **1 RTT**:

Paso	Responsable	Acción	RTT
1	Cliente	Establece la conexión con el servidor (SYN, SYN+ACK, ACK)	**1**
2	Cliente	Envía Request pidiendo la página (`GET /index.html HTTP/1.1`)	1 (compartido con el 3)
3	Servidor	Envía Response con el contenido (`HTTP/1.1 200 OK` + HTML)	cierra el RTT
4	Servidor	Cierra la conexión	0 (FIN)
5–8		Se repiten por cada una de las 10 imágenes	2 c/u

Precisión 2 — la lectura del cuerpo HTML. El paso 3 dice "envía response con el contenido de la página". Lo que la oficial da por supuesto es que el cliente, al recibir el HTML, **lo parsea**, ve las 10 imágenes referenciadas por `<img src="imagen1.png">`, y dispara 10 peticiones, cada una repitiendo 1–4.

Respuesta redactada, lista para copiar en el examen:

> **a)** En HTTP **no persistente**, por cada objeto (el HTML y cada una de las 10 imágenes) el cliente ejecuta **cuatro pasos**: (1) establece una conexión TCP con el servidor, (2) envía un request pidiendo el objeto, (3) el servidor envía un response con el contenido del objeto, (4) el servidor cierra la conexión. El primer objeto es la página HTML; al recibirla y parsearla, el cliente descubre las 10 imágenes referenciadas y repite los pasos 1–4 **una vez por cada imagen**. En total son **11 objetos × 4 pasos = 44 pasos**, con **11 conexiones TCP distintas** y **22 RTT** (2 RTT por objeto: 1 de handshake TCP + 1 de request/response).
>
> **b)** En HTTP **persistente** se establece **una sola** conexión TCP y por ella se piden los 11 objetos uno tras otro: **1 RTT** de handshake + **11 RTT** de request/response = **12 RTT**, contra los 22 del no persistente. La mejora es real cuando el navegador trabaja **secuencialmente**, porque se eliminan los `N − 1 = 10` handshakes. **Pero no siempre mejora.** Si el navegador abre conexiones **en paralelo** (que es lo que hacen los reales) y los objetos vienen de **varios servidores**, los handshakes pasan todos al mismo tiempo: el no persistente baja a **4 RTT** y el persistente queda en **4 RTT**, o sea, la ventaja del persistente **desaparece**. El persistente es claramente superior con un solo servidor (3 RTT contra 4, con paralelo, y 12 contra 22 sin paralelo), pero con varios servidores y paralelismo hay empate.
>
> **Nota sobre mi respuesta contra la oficial:** la oficial dice "puede resultar más performante", que es la respuesta honesta y matizada, y agrega un argumento que repito: **"si el servidor nos limita por**

> **IP"**, o sea, que el navegador no puede paralelizar porque el servidor impone un límite por IP. Ese es el mejor argumento del apartado b).

La cuenta de RTT completa, a la vista

Este es el corazón del asunto y lo que la oficial no cuantifica:

```
========== NO PERSISTENTE, 1 SERVIDOR, SECUENCIAL ==========

objeto    |  TCP handshake  |  request/resp  |  RTT del objeto
----------+-----------------+----------------+-----------------
  HTML    |       1         |        1       |      2
  img 1   |       1         |        1       |      2
   ...    |       1         |        1       |      2
  img 10  |       1         |        1       |      2
----------+-----------------+----------------+-----------------
TOTAL     |      11         |       11       |    22 RTT


========== PERSISTENTE, 1 SERVIDOR, SECUENCIAL ==========

objeto    |  TCP handshake  |  request/resp  |  RTT del objeto
----------+-----------------+----------------+-----------------
  HTML    |       1         |        1       |      2
  img 1   |       0         |        1       |      1
   ...    |       0         |        1       |      1
  img 10  |       0         |        1       |      1
----------+-----------------+----------------+-----------------
TOTAL     |       1         |       11       |    12 RTT


========== PERSISTENTE, PARALELO (6 conexiones), 1 SERVIDOR ==========

objeto    |  TCP handshake  |  request/resp  |
----------+-----------------+----------------+
  HTML    |       1         |        1       |
10 imgs   |       1         |        1       |   ← todos a la vez, en paralelo
----------+-----------------+----------------+
TOTAL                                                    4 RTT

... y si además el navegador REUTILIZA la conexión del HTML para las
10 imágenes (que es lo que hace con las 6 conexiones por host):

TOTAL = 1 (handshake) + 1 (HTML) + 1 (las 10 imágenes, en paralelo) = 3 RTT
```

Interpretación, que es lo que hay que escribir: el mecanismo por el que el persistente mejora es **eliminar handshakes**, no "mandar más rápido". Cada handshake eliminado vale 1 RTT: con 11 objetos y 1 servidor se eliminan 10. En el límite de la conexión paralela el handshake ya **no es el cuello de botella** y lo que manda es el **número de RTT de request/response**, y para eso sirven **HTTP/2 y HTTP/3** (Clase 9).

⚠️ Errores típicos en este problema

- **Olvidar que el paso 1 es TCP.** La oficial lo abstrae, y está bien para el apartado a), pero si te piden RTTs **no** podés contarlo como 1 RTT de HTTP.

- **"El cliente pide el HTML y después pide las 10 imágenes"** sin notar que en no persistente **cada** petición abre conexión. Ese es el error conceptual central.
- **Contar 11 RTT en no persistente.** Son 22.
- **Decir "el persistente siempre es mejor".** El enunciado dice **"discuta"**. Hay que dar el caso en que **no** mejora.

3.2 Examen 2009-07, Problema 2 — el `getMailServer()` como puente

> **Fuente:** Examen `eirc0907` / `sirc0907` — 29 de julio de 2009, Problema 2.

Este problema **une HTTP con DNS** y por eso lo desarrollamos completo en la **Clase 14**, donde se implementa `getMailServer()` con sockets UDP. Lo que aporta acá es el **lado HTTP**.

> **Apartado b)** Implementar `bool isSpam(string ipAddress, string subject)` que devuelve `true` si naivespamming considera que el correo es spam.

La solución oficial escribe:

```
Socket s = new Socket("www.naivespamming.org", 80);
if (!s.connect()) return false;
s.out.write(
  "GET isspam.cgi?sender=" + ipAddress +
  "&subject=" + queryStringStuff(subject) +
  " HTTP/1.1\r\n\r\n");
string response = s.in.readLine();
s.close();
string[] result = response.split();
return result[1] == "200";
```

Análisis, con lo que aprendimos hoy:

1. **El `GET` está mal formado como HTTP/1.1.** Un mensaje **válido** sería:

```
GET /isspam.cgi?sender=203.0.113.7&subject=Hola HTTP/1.1
Host: www.naivespamming.org
User-Agent: FING-SpamFilter/1.0
Accept: text/plain
Connection: close
```

Falta el `Host`, que es obligatoria en 1.1: sin ella, un servidor con *vhosts* no sabe qué sitio querés. Y usa un **path relativo** (`isspam.cgi`) en vez de absoluto (`/isspam.cgi`). El `\r\n\r\n` final sí es correcto: es la línea en blanco que cierra las cabeceras, tal como vimos en §1.3. 2. **El parseo es frágil.** `response.split()` separa por espacios y toma `result[1]`, que es el **código de estado**. Funciona, pero se rompe si hay redirección. **Lo robusto** es parsear la **primera línea** de la respuesta. 3. **El `return result[1] == "200"` tiene la semántica invertida** respecto del enunciado, que dice explícitamente que **404 significa "es spam"** y **200 significa "es legítimo"**. La oficial devuelve `true` con 200, o sea, marca

como spam un correo que el servidor dijo que es legítimo. **Es un error real de la solución publicada.** Lo correcto:

```
código == 404  →  es spam     →  return true
código == 200  →  no es spam  →  return false
otro / error   →  no se sabe  →  return false  (fail-open: mejor dejar
                    pasar un correo que trabar todo el servidor)
```

> **Nota explícita:** el enunciado y la oficial se contradicen. Si te pasa en un examen, **respetá el enunciado** y **señalá la contradicción** por escrito. Eso suma puntos: demuestra que leíste los dos documentos.

Lo que aporta esta pregunta al tema de hoy: los **códigos de estado** importan de verdad. Un 404 no es un error técnico: es **información de negocio** para el servidor de correo. **HTTP no solo transporta contenido, transporta semántica**, y un `4xx` / `2xx` puede ser exactamente el dato que la aplicación necesita.

PARTE 4 — CIERRE

📌 Resumen

1. **Cliente/servidor** = roles **asimétricos**: el servidor **espera**, el cliente **inicia**. La asimetría elimina el problema de "quién llama".
2. **Peer-to-peer** = todos son cliente y servidor a la vez, sin punto central. Escala con los pares, pero es mucho más complejo.
3. **HTTP no es** un protocolo de transporte, **ni** la Web, **ni** tiene estado, **ni** es seguro. Es solo un formato de mensajes de aplicación sobre TCP.
4. El mensaje HTTP tiene **3 partes**: línea de petición (o de estado) + cabeceras + **línea en blanco obligatoria** + cuerpo.
5. `Host` es **obligatoria** en HTTP/1.1; `Content-Length` y `Content-Type` son las que hacen que la respuesta sea interpretable.
6. **No persistente**: 2 RTT por objeto. **Persistente**: 1 RTT de handshake + 1 RTT por objeto. Con 11 objetos: **22 RTT vs. 12 RTT** en un solo servidor.
7. Con **paralelo** y varios servidores, el persistente **empata** (4 RTT): el handshake deja de ser el cuello de botella.
8. **304 Not Modified**: no cambia el ETag, no viaja cuerpo, ahorra ~96 % de la transferencia. Es **validación condicional** con `If-None-Match` / `If-Modified-Since`.
9. **401** = "no sé quién sos, mandá credenciales". **403** = "sé quién sos y no podés". No son sinónimos.
10. Más del **80 %** del tiempo de carga de una página es **espera**, no transferencia. Cambiar de protocolo no acelera la red: **elimina esperas**.
11. **Cookies**: el estado vive **en el cliente** (`Set-Cookie` del servidor, `Cookie` del cliente). Es el precio de que el servidor sea sin estado y pueda escalar. `RFC 6265` las define; `SameSite` / `HttpOnly` las aseguran.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué el diseño sin estado de HTTP es, en el fondo, una decisión de escalabilidad? Explicá el argumento con un balanceador de carga.

<details>
<summary>▶ Respuesta</summary>
</details>

2. ¿Cuántos RTT cuesta una página con 11 objetos en HTTP no persistente? Y en persistente? Mostrá las cuentas.

<details>
<summary>▶ Respuesta</summary>
</details>

3. ¿En qué caso el HTTP persistente **no** mejora la performance? Justificá.

▶ Respuesta

4. ¿Qué diferencia hay entre `Set-Cookie` y `Cookie`? ¿En qué dirección viaja cada una?

▶ Respuesta

5. ¿Por qué un **304** no tiene `Content-Length` ni cuerpo? ¿Qué se ahorró?

▶ Respuesta

6. ¿Cuál es la diferencia entre `401` y `403`? ¿Por qué importa?

▶ Respuesta

7. Un cliente descarga una página de 11 objetos con RTT de 100 ms, ancho de banda de 10 Mbps y objetos de 50 KB cada uno, en HTTP no persistente. ¿Qué porcentaje del tiempo total se va en **espera**?

▶ Respuesta

8. En la solución oficial del `2009-07 Problema 2`, la función `isSpam()` devuelve `true` cuando el código HTTP es 200. ¿Es correcto? ¿Qué haría bien?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1007` / `sirc1007`** — Pregunta 1 completa (8 pts, HTTP no persistente + las 10 imágenes)
- **Examen `eirc0907` / `sirc0907`** — Problema 2 (30 pts): cliente anti-spam vía HTTP; apartados b) y c)
- **Examen `eirc1002` / `sirc1002`** — Pregunta 1a (8 pts): qué es el DNS y su función (desarrollada en la Clase 10)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.1 (modelo cliente/servidor) y §7.3 (HTTP: los dos tipos de conexión, los mensajes, el cálculo de RTT)
- **RFC 9110** — HTTP Semantics (2022, *Internet Standard*; sustituye a RFC 2616) · **RFC 9112** — HTTP/1.1 (2022, sintaxis)
- **RFC 9113** — HTTP/2 (2022; obsoleta RFC 7540) · **RFC 9114** — HTTP/3 (2022, sobre QUIC)
- **RFC 6265** — HTTP State Management Mechanism (cookies) · **RFC 7838** — Alt-Svc (2016): cómo se negocia HTTP/3
- **Clase siguiente:** 9 — HTTP a fondo: mensajes, cookies, caché y versiones